

DSP Laboratory Experiment

Fixed-Point Quantization and SQNR Analysis using MATLAB and Verilog

Aslin Garvasis
EE24BTECH11008

January 26, 2026

Aim

To study the effect of fixed-point quantization on a sinusoidal signal, analyze quantization error, compute the Signal-to-Quantization-Noise Ratio (SQNR), and verify the results using MATLAB and Verilog.

Theory

Signal Sampling

A continuous-time sinusoidal signal is given by

$$x(t) = A \sin(2\pi ft)$$

where $A = 2$ and $f = 1$ kHz. Sampling the signal at $F_s = 48$ kHz gives

$$x[n] = A \sin\left(2\pi f \frac{n}{F_s}\right)$$

Fixed-Point Representation (Q-Format)

Fixed-point numbers are represented using a fixed number of integer and fractional bits:

$$Q(I, F)$$

where I is the number of integer bits and F is the number of fractional bits. The formats used are Q(2,14), Q(4,12), and Q(8,4).

Quantization Using Truncation

Quantization is performed using truncation to accurately model hardware behavior:

$$x_q[n] = \frac{\text{fix}(x[n] \cdot 2^F)}{2^F}$$

The quantization error is

$$e[n] = x[n] - x_q[n]$$

Signal-to-Quantization-Noise Ratio (SQNR)

$$\text{SQNR} = \frac{E[x^2]}{E[e^2]} \quad , \quad \text{SQNR(dB)} = 10 \log_{10} \left(\frac{E[x^2]}{E[e^2]} \right)$$

Procedure

MATLAB

1. Generate a sampled 1 kHz sine wave.
2. Perform fixed-point quantization using truncation.
3. Plot original, quantized, and error signals.
4. Compute SQNR.

Verilog

1. Generate sine wave samples using Verilog.
2. Quantize using fixed-point arithmetic.
3. Export data to text files.
4. Analyze and plot results in MATLAB.

Major Operations in MATLAB Implementation

- The signal parameters such as frequency (f), sampling frequency (F_s), amplitude (A), and time period (T) are defined to generate a 1 kHz sinusoidal signal sampled at 48 kHz.
- A discrete-time vector is created using the sampling interval $1/F_s$, and only five cycles of the signal are selected for visualization to clearly observe quantization effects.

- The continuous-amplitude sinusoidal signal is generated using the sine function and plotted using stem plots to emphasize the discrete-time nature of the sampled signal.
- Fixed-point quantization is performed using the `fix` function, which truncates the scaled signal values. This models hardware-style truncation commonly used in fixed-point DSP implementations.
- Three different Q-formats are implemented: Q(2,14), Q(4,12), and Q(8,4), representing high, medium, and low fractional precision respectively.
- The quantized integer values are rescaled back to real values by dividing with the corresponding power of two, allowing direct comparison with the original signal.
- Quantization error is computed as the difference between the original signal and the quantized signal for each Q-format.
- Original and quantized signals are plotted together to visually compare the effects of different fixed-point precisions.
- Separate error plots are generated to analyze how the magnitude and nature of quantization error change with reduced fractional resolution.
- The Signal-to-Quantization-Noise Ratio (SQNR) is computed using the ratio of mean squared signal power to mean squared error power and expressed in decibels.
- Finally, the SQNR results for different Q-formats are tabulated for easy comparison and interpretation.

MATLAB Code

```

1  f  = 1000;
2  T  = 1/f;
3  Fs = 48000;
4  A  = 2;
5  t  = 0:1/Fs:1-1/Fs;
6  N  = round(5*T*Fs);
7
8  x  = A*sin(2*pi*f*t);
9
10 figure;
11 stem(t(1:N), x(1:N), 'filled');
12 xlabel('Time (s)');

```

```

13 ylabel('Amplitude');
14 title('Sampled_Signal_(5_Cycles_of_1_kHz_Sine_Wave)');
15 grid on;
16 Q2_14_int = fix(x * 2^14);
17 Q4_12_int = fix(x * 2^12);
18 Q8_4_int = fix(x * 2^4);
19 x_Q2_14 = Q2_14_int / 2^14;
20 x_Q4_12 = Q4_12_int / 2^12;
21 x_Q8_4 = Q8_4_int / 2^4;
22 e_Q2_14 = x - x_Q2_14;
23 e_Q4_12 = x - x_Q4_12;
24 e_Q8_4 = x - x_Q8_4;
25 figure;
26 stem(t(1:N), x(1:N), 'b','filled'); hold on;
27 stem(t(1:N), x_Q2_14(1:N), 'r');
28 xlabel('Time_(s)');
29 ylabel('Amplitude');
30 title('Original_vs_Quantized_Signal_(Q(2,14))');
31 legend('Original','Quantized');
32 grid on;
33 figure;
34 stem(t(1:N), x(1:N), 'b','filled'); hold on;
35 stem(t(1:N), x_Q4_12(1:N), 'g');
36 xlabel('Time_(s)');
37 ylabel('Amplitude');
38 title('Original_vs_Quantized_Signal_(Q(4,12))');
39 legend('Original','Quantized');
40 grid on;
41 figure;
42 stem(t(1:N), x(1:N), 'b','filled'); hold on;
43 stem(t(1:N), x_Q8_4(1:N), 'm');
44 xlabel('Time_(s)');
45 ylabel('Amplitude');
46 title('Original_vs_Quantized_Signal_(Q(8,4))');
47 legend('Original','Quantized');
48 grid on;
49 figure;
50 stem(t(1:N), e_Q2_14(1:N), 'filled');
51 xlabel('Time_(s)');
52 ylabel('Error');
53 title('Quantization_Error_(Q(2,14))');

```

```

54 grid on;
55 figure;
56 stem(t(1:N), e_Q4_12(1:N), 'filled');
57 xlabel('Time(s)');
58 ylabel('Error');
59 title('Quantization_Error(Q(4,12))');
60 grid on;
61 figure;
62 stem(t(1:N), e_Q8_4(1:N), 'filled');
63 xlabel('Time(s)');
64 ylabel('Error');
65 title('Quantization_Error(Q(8,4))');
66 grid on;
67 SQNR_Q2_14 = mean(x.^2)/mean(e_Q2_14.^2);
68 SQNR_Q4_12 = mean(x.^2)/mean(e_Q4_12.^2);
69 SQNR_Q8_4 = mean(x.^2)/mean(e_Q8_4.^2);
70 SQNR_dB = 10*log10([SQNR_Q2_14 SQNR_Q4_12 SQNR_Q8_4]);
71 formats = {'Q(2,14)', 'Q(4,12)', 'Q(8,4)'};
72 SQNR_table = table(formats, SQNR_dB, ...
73     'VariableNames', {'Q_Format', 'SQNR_dB'});
74 disp(SQNR_table);

```

Major Operations in Verilog Implementation

- The Verilog code is written as a testbench-style module to simulate signal generation and fixed-point quantization rather than hardware synthesis.
- Signal parameters such as frequency, sampling frequency, amplitude, and time period are initialized using real variables to match the MATLAB implementation.
- The number of samples corresponding to five cycles of the sinusoidal signal is computed to ensure identical signal duration in both MATLAB and Verilog.
- A for-loop is used to generate discrete-time samples of the sinusoidal signal using the built-in \$sin function.
- Fixed-point quantization is implemented by scaling the signal with powers of two and truncating the result using \$rtoi, which is equivalent to MATLAB's fix operation.
- Three Q-formats, Q(2,14), Q(4,12), and Q(8,4), are implemented to analyze the effect of different fractional precisions.

- The quantized integer values are rescaled back to real numbers to compute quantization error accurately.
- Quantization error is calculated as the difference between the original and quantized signal values for each Q-format.
- The original signal, quantized signal, and quantization error values are written to text files using file I/O system tasks for further analysis.
- MATLAB is used to read the generated text files and plot the waveforms, enabling direct comparison between MATLAB-only and Verilog-based implementations.
- The close agreement between MATLAB and Verilog results confirms that the Verilog model accurately represents fixed-point quantization behavior.

Verilog Code

```

1  'timescale 1ns/1ps
2  module quantization_tb;
3      integer i;
4      real f;
5      real T;
6      real Fs;
7      real A;
8      real t;
9      integer N;
10     real x;
11     integer Q2_14;
12     integer Q4_12;
13     integer Q8_4;
14     real x_Q2_14;
15     real x_Q4_12;
16     real x_Q8_4;
17     real e_Q2_14;
18     real e_Q4_12;
19     real e_Q8_4;
20     integer f_org, f_q14, f_q12, f_q4;
21     initial begin
22         f  = 1000.0;
23         T  = 1.0 / f;
24         Fs = 48000.0;
25         A  = 2.0;

```

```

26     N = $rtoi(5.0 * T * Fs);
27     f_org = $fopen("original.txt","w");
28     f_q14 = $fopen("q14.txt","w");
29     f_q12 = $fopen("q12.txt","w");
30     f_q4 = $fopen("q4.txt","w");
31     for (i = 0; i < N; i = i + 1) begin
32         t = i / Fs;
33         x = A * $sin(2.0 * 3.141592 * f * t);
34         Q2_14 = $rtoi(x * (1<<14));
35         Q4_12 = $rtoi(x * (1<<12));
36         Q8_4 = $rtoi(x * (1<<4));
37         x_Q2_14 = Q2_14 / (1.0 * (1<<14));
38         x_Q4_12 = Q4_12 / (1.0 * (1<<12));
39         x_Q8_4 = Q8_4 / (1.0 * (1<<4));
40         e_Q2_14 = x - x_Q2_14;
41         e_Q4_12 = x - x_Q4_12;
42         e_Q8_4 = x - x_Q8_4;
43         $fdisplay(f_org, "%f", x);
44         $fdisplay(f_q14, "%f□%f", x_Q2_14, e_Q2_14);
45         $fdisplay(f_q12, "%f□%f", x_Q4_12, e_Q4_12);
46         $fdisplay(f_q4, "%f□%f", x_Q8_4, e_Q8_4);
47     end
48     $fclose(f_org);
49     $fclose(f_q14);
50     $fclose(f_q12);
51     $fclose(f_q4);
52     $finish;
53 end
54 endmodule

```

Results

SQNR Results (MATLAB)

Table 1: SQNR Results using MATLAB

Q-Format	SQNR (dB)
Q(2,14)	90.75
Q(4,12)	79.17
Q(8,4)	32.29

SQNR Results (Verilog)

Table 2: SQNR Results using Verilog

Q-Format	SQNR (dB)
Q(2,14)	91.737
Q(4,12)	79.708
Q(8,4)	31.744

MATLAB Figures

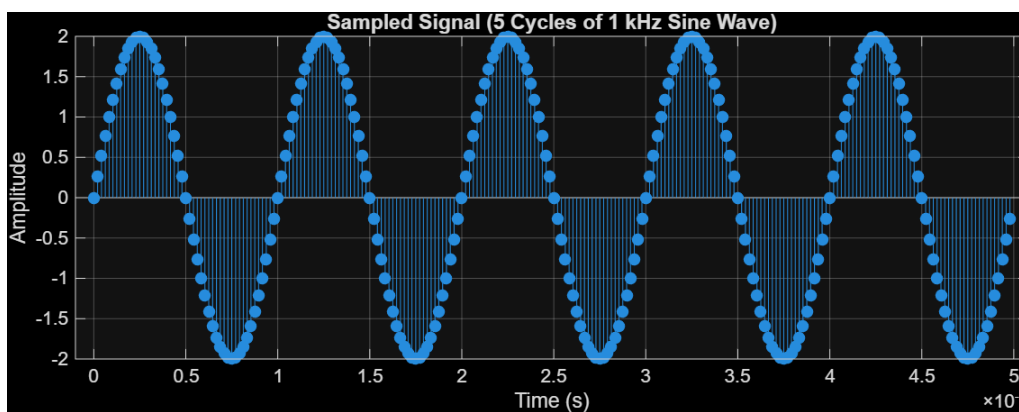


Figure 1: Sampled 1 kHz sinusoidal signal (5 cycles) generated using MATLAB

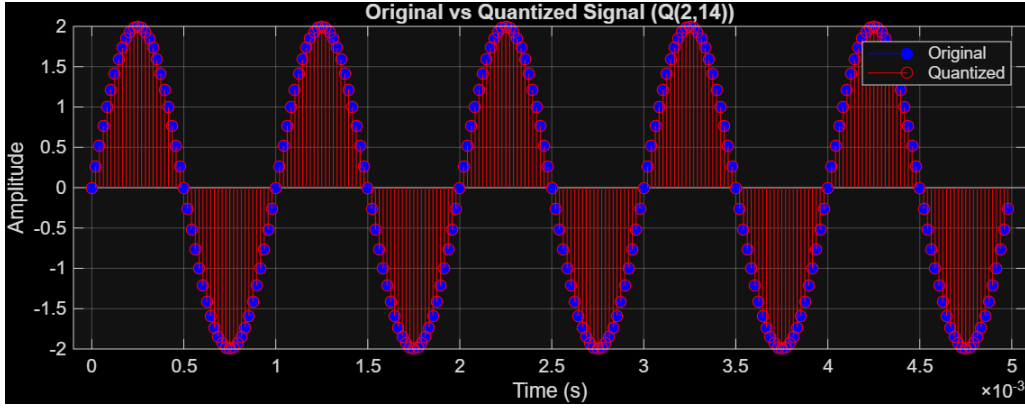


Figure 2: Original and fixed-point quantized signal using Q(2,14) format (MATLAB)

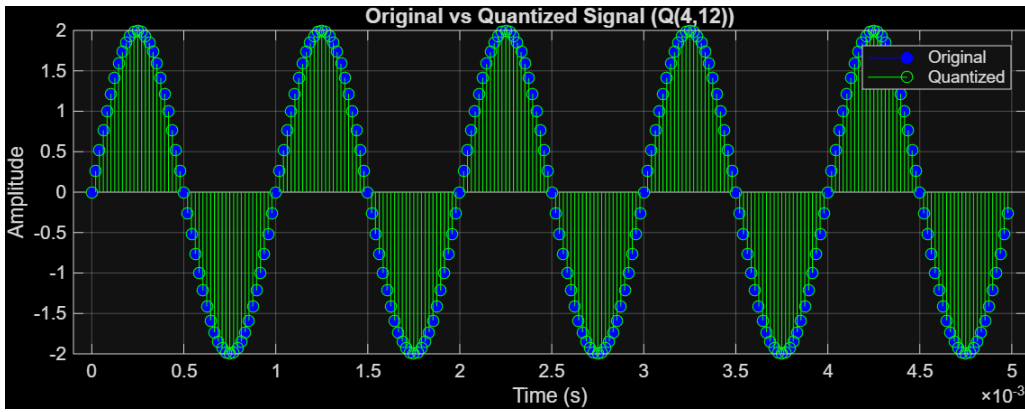


Figure 3: Original and fixed-point quantized signal using Q(4,12) format (MATLAB)

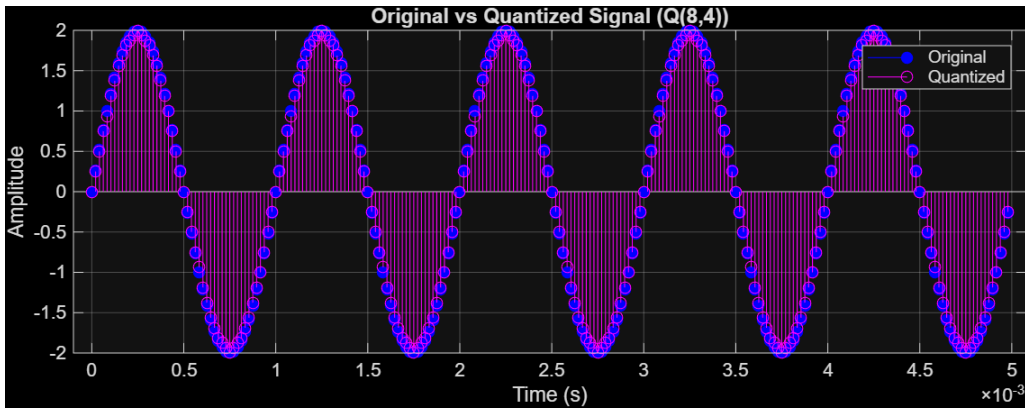


Figure 4: Original and fixed-point quantized signal using Q(8,4) format (MATLAB)

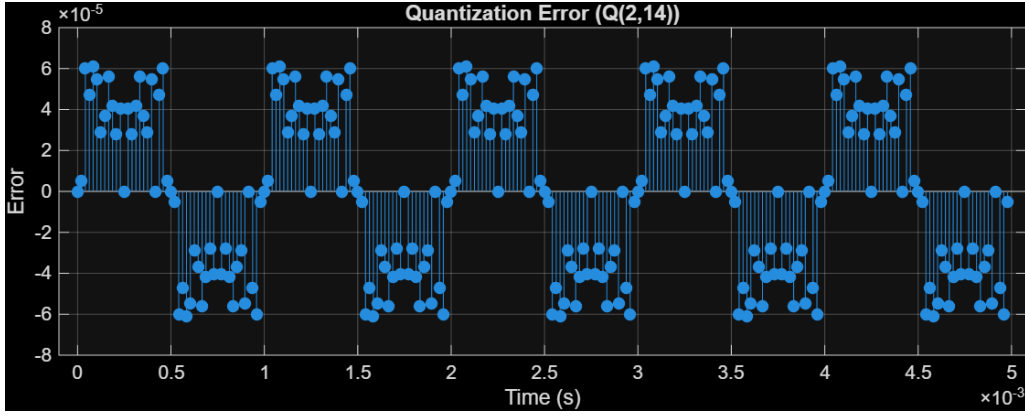


Figure 5: Quantization error for Q(2,14) fixed-point representation (MATLAB)

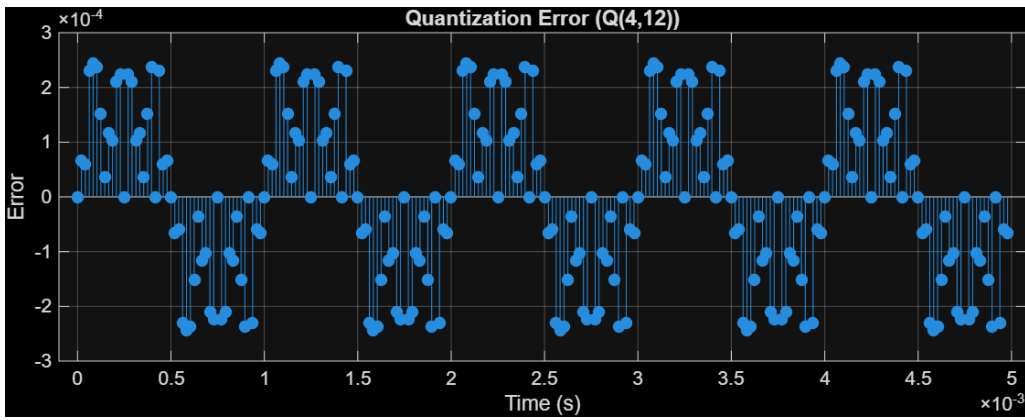


Figure 6: Quantization error for Q(4,12) fixed-point representation (MATLAB)

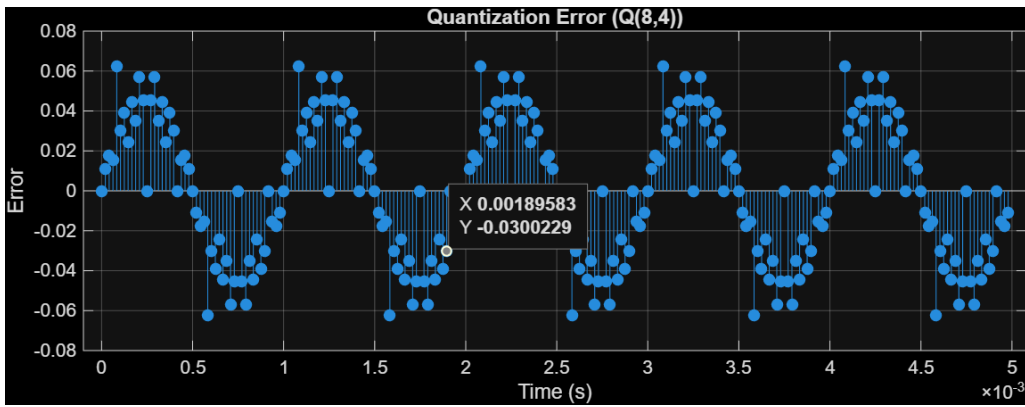


Figure 7: Quantization error for Q(8,4) fixed-point representation (MATLAB)

Verilog Figures

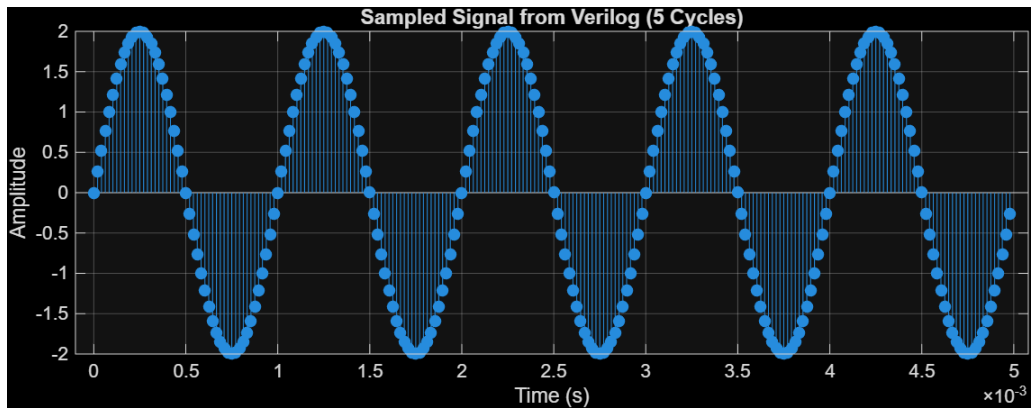


Figure 8: Sampled 1 kHz sinusoidal signal generated using Verilog

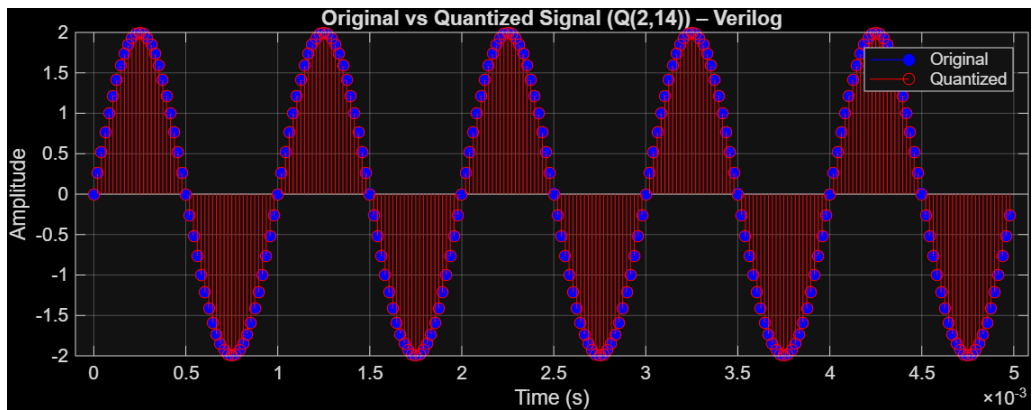


Figure 9: Original and fixed-point quantized signal using Q(2,14) format (Verilog)

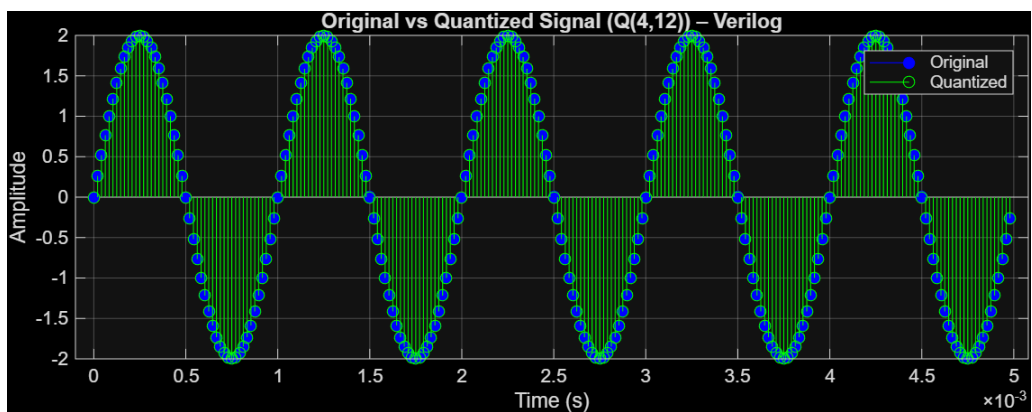


Figure 10: Original and fixed-point quantized signal using Q(4,12) format (Verilog)

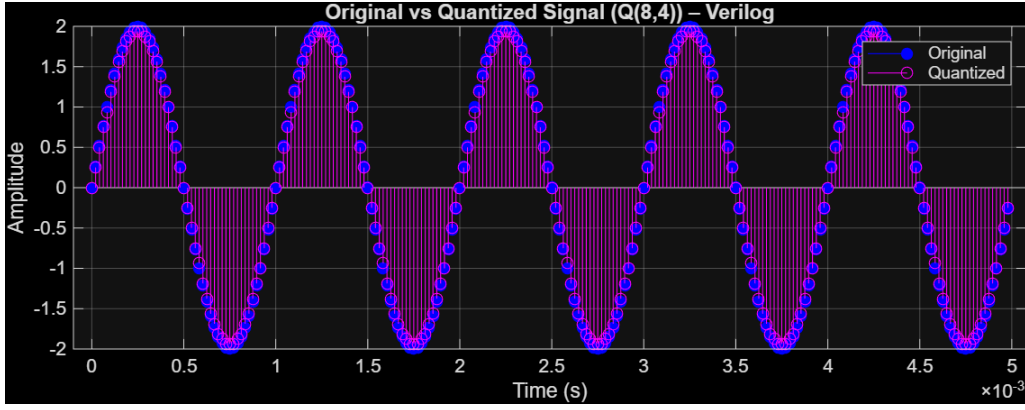


Figure 11: Original and fixed-point quantized signal using Q(8,4) format (Verilog)

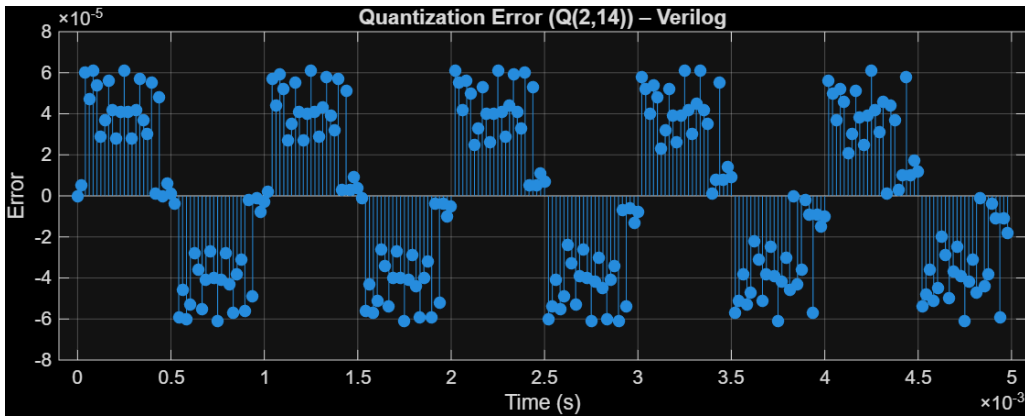


Figure 12: Quantization error for Q(2,14) obtained from Verilog simulation

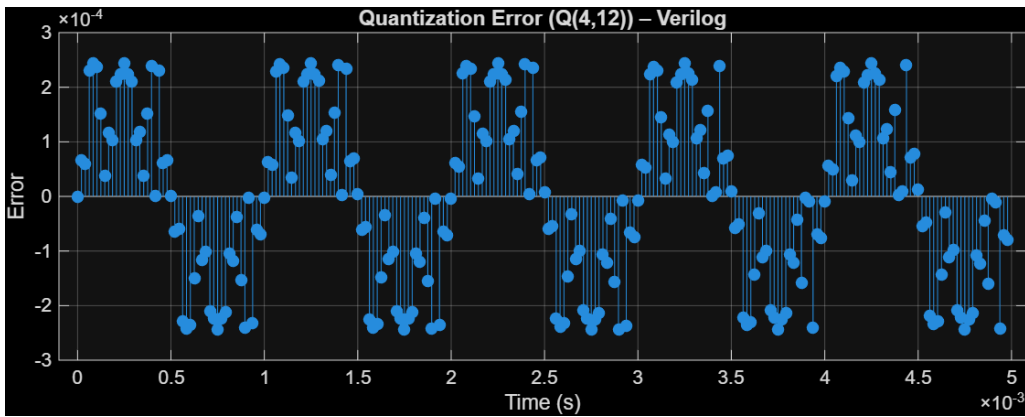


Figure 13: Quantization error for Q(4,12) obtained from Verilog simulation

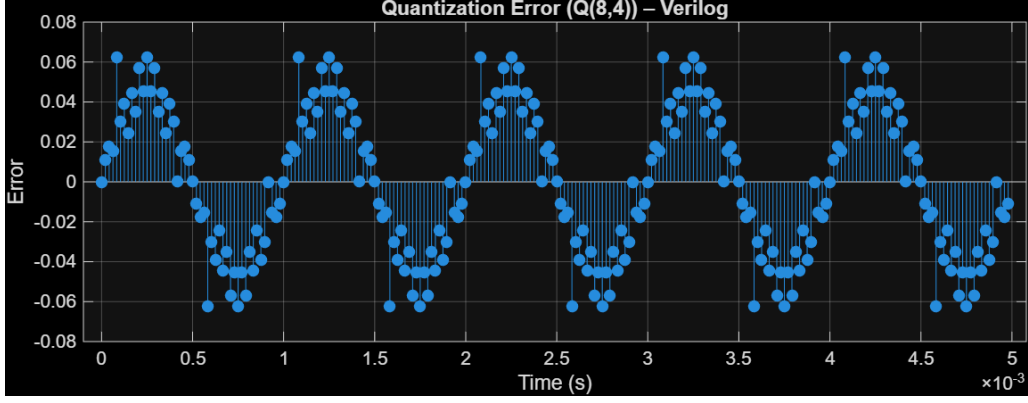


Figure 14: Quantization error for Q(8,4) obtained from Verilog simulation

Comparison and Discussion

- The MATLAB and Verilog results show close agreement in both waveform plots and SQNR values, confirming the correctness of the fixed-point quantization model implemented in both environments.
- For the high-precision format **Q(2,14)**, the quantized signal almost overlaps the original signal in both MATLAB and Verilog plots. The corresponding error plots exhibit very small, noise-like variations, indicating minimal quantization distortion due to the large number of fractional bits.
- The **Q(4,12)** format represents a middle ground between precision and hardware complexity. In this case, slight staircase effects become visible in the quantized signal, and the error magnitude increases compared to Q(2,14). However, the overall waveform shape is still well preserved, demonstrating that Q(4,12) offers a good trade-off between accuracy and resource usage.
- For the low-precision format **Q(8,4)**, significant staircase distortion is observed in the quantized signal. The corresponding error plots show large amplitude variations and increased correlation with the input signal, indicating severe loss of fractional resolution.
- The quantization error increases monotonically as the number of fractional bits decreases. This behavior is consistent across both MATLAB and Verilog implementations, validating the theoretical relationship between quantization step size and error magnitude.
- The SQNR values decrease significantly as the fractional precision is reduced. MATLAB results show SQNR values of 90.75 dB, 79.17 dB, and 32.29 dB for Q(2,14), Q(4,12), and Q(8,4) respectively, while Verilog results closely follow the same trend.

- Minor differences between MATLAB and Verilog SQNR values arise due to numerical precision, floating-point evaluation in MATLAB, and simulation-level truncation effects in Verilog. These differences are small and do not affect the overall conclusions.
- Overall, the comparison confirms that higher fractional precision leads to lower quantization error and higher SQNR, while reduced precision increases distortion but may be acceptable in hardware-constrained applications.

Conclusion

The experiment successfully demonstrates the effect of fixed-point quantization on signal quality. Higher fractional precision results in lower quantization error and higher SQNR.